Creating a toadally_safe Password

Liran Ma and Taylor Griffin
Department of Computer Science
Texas Christian University

Copyright © 2023 Liran Ma, Texas Christian University.
The development of this document is partially funded by the National Science Foundation's and SaTC program under award No. DGE-1912755. and Permission is granted to copy, distribute and/or modify this document under the terms of the GNU Free Documentation License, Version 1.2 or any later version published by the Free Software Foundation. A copy of the license can be found at http://www.gnu.org/licenses/fdl.html.

Overview

Passwords serve as the first line of defense when it comes to securing our online accounts. In today's digital landscape, where cyber threats loom at every corner, it is crucial to understand the significance of creating strong and secure passwords. By doing so, we fortify our accounts against malicious attackers who seek to exploit our personal information, financial assets, and online presence.

In this lab, we will evaluate the strength of passwords and discover how to improve them.

To do this, we will utilize Docker and toadally_safe, a Password Strength Estimator Web Application that implements the zxcvbn Low-Budget Password Strength Estimation library.

Contents

1 Introduction **2**

2 Exploring the zxcvbn Library **3**

3 Testing Password Strength with toadally_safe **4**

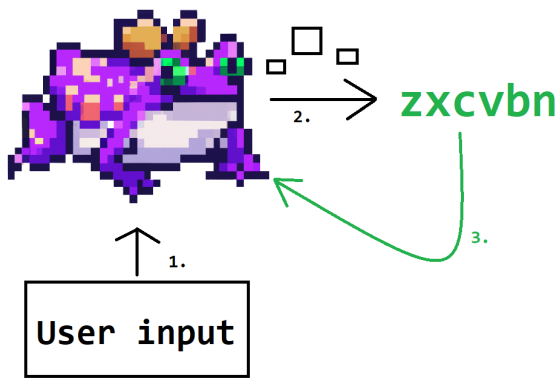
4 Enhancing Password Security **6**

5 Sources and Additional Reading **9**

6 Troubleshooting **10**

1 Introduction

toadally_safe is a Password Strength Estimator Web Application that employs the zxcvbn Low-Budget Password Strength Estimation library. This application is designed to assist users in evaluating the strength of their passwords and provides guidance on how to enhance them.



As you enter or modify your password, toadally_safe provides real-time feedback, allowing you to instantly view the impact of your changes to optimize your password strength quickly. It provides users with security metrics such as password scoring, improvement suggestions, warnings, and estimated time to crack given the power of different computing resources. toadally_safe offers the convenience of processing files containing passwords, enabling you to assess the strength of numerous passwords simultaneously.

1.1 Lab Prep (Docker)

Before we dive into using toadally_safe, it is necessary to set up Docker. Docker is a platform that allows for the creation and management of containers, enabling seamless deployment of applications.

To prepare for this lab, follow these steps to set up Docker:

Install Docker: Visit the official Docker website (https://www.docker.com/) and download the appropriate version of Docker for your operating system. Follow the installation instructions provided.

Verify Docker installation: Once Docker is installed, open a terminal or command prompt and enter the following command:

```
docker --version
# This command will display the Docker version if the installation was successful.
```

Next, follow the code to create a new folder and clone the toadally_safe repository:

```
mkdir password_estimation
cd password_estimation
git clone https://github.com/imtaylorgriffin/toadally_safe.git
# Use ``git pull'' if you have cloned the project before
```

1.1.1 Explore the files in the toadally_safe folder. Describe how toadally_safe implements zxcvbn into the application

2 Exploring the zxcvbn Library

toadally_safe utilizes zxcvbn Low-Budget Password Strength Estimation library (https://github.com/dropbox/zxcvbn) in this section of the lab, we'll look at zxcvbn's functionality and capabilities. The library is well-known for its ability to assess the strength of passwords based on a variety of factors. Let's look at the it's main features:

2.1 Functionality of zxcvbn

The zxcvbn library offers several key functionalities that contribute to its accurate password strength estimation. Let's explore these functionalities in more detail:

Common Pattern Matching:

```
zxcvbn incorporates pattern matching techniques to identify common password
patterns used by users.

It recognizes and weighs approximately 30,000 commonly used passwords,
including variations with substitutions and character transformations.

To enhance its accuracy, zxcvbn incorporates data from various sources such as
the US census, Wikipedia, and popular US television shows and movies.

By comparing user-provided passwords to this extensive list, zxcvbn
can assess the strength of passwords based on their similarity to known
weak passwords.
```

Conservative Estimation:

```
zxcvbn employs conservative estimation methods to evaluate password
strength accurately.

It takes into account the worst-case scenario for password guessing attacks
```

```
and provides a reliable estimation of the time it would take to crack a password.
```

Keyboard and Sequence Matching:

```
zxcvbn also evaluates passwords for other common patterns that make them
vulnerable to attacks.

It checks for sequences of characters (e.g., "abcd"), keyboard patterns
(e.g., "qwertyuiop"), and repeated characters (e.g., "aaa").

Additionally, it recognizes common password patterns in "l33t speak"
(e.g., "p4ssw0rd") and adjusts their impact on password strength evaluation.
```

Integration with Attacker Data:

```
zxcvbn is trained using password leaks and dictionaries commonly used by
attackers in real-world scenarios.

By leveraging this data, zxcvbn can predict the effectiveness of today's
best guessing attacks within the context of an online attack.

It achieves this by checking the minimum rank of a password across multiple
frequency-ranked lists and using light pattern matching techniques.
```

By combining pattern matching, conservative estimation, recognition of common names and words, and integration with attacker data, zxcvbn offers a comprehensive approach to password strength estimation. This allows the toadally_safe web application to provide users with accurate assessments of their password strength and actionable recommendations for improving security.

3 Testing Password Strength with toadally_safe

In this section of the lab, we'll look at how to use the toadally_safe web application to assess password strength. Following these steps will provide you with information about the efficacy of your passwords as well as tips on how to strengthen them.

After cloning, cd into the new toadally_safe folder and start the application:

```
cd toadally_safe
docker-compose up
# Starts all containers, make sure you're in the toadally_safe folder
```

Following that, head over to your browser and type:

```
http://localhost:3000
```

You should be greeted with the user interface shown in **Figure 1: a**

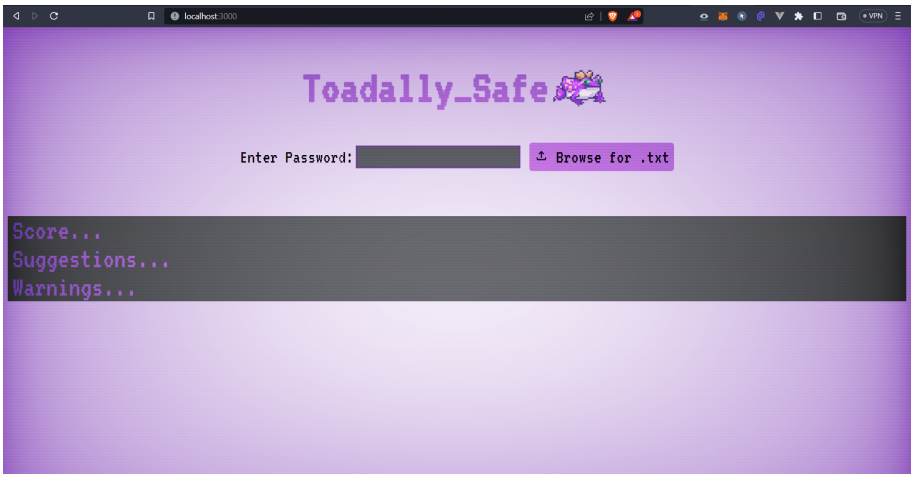


Figure 1: a

3.1 Using toadally_safe

toadally_safe has two input options, you can type in a password inside the field, or you can use the *Browse for .txt* button to use a .txt file with new-line separated passwords.

In the root directory, there are included example password files: *passwords.txt* and *passwords2.txt*

3.1.1 (Optional) You can create a file and fill it with whatever passwords you'd like.

3.2 Testing different Passwords

On the toadally_safe web application's homepage, locate the password input field.

Enter a password of your choice in the input field.

As you type, the application will provide real-time feedback and metrics regarding the strength of your password.

3.2.1 Enter "password" (without quotation marks) make note of the output.

3.2.2 Visit norton's Password Generator and generate a new password (https://my.norton.com/extspa/passwordmanager?path=pwd-gen) Paste this new password into toadally_safe. How does this compare to the output before?

3.3 File Processing

Explore the file processing feature of toadally_safe to assess the strength of multiple passwords simultaneously.

Using your own file containing a list of passwords separated by new lines, or the provided *passwords.txt* and *passwords2.txt*, upload the file using the *Browse for .txt* button in toadally_safe.

3.3.1 Evaluate the results provided by the application, including the strength assessment and any improvement suggestions. What kind of passwords give better results?

4 Enhancing Password Security

In this section of the lab, we will focus on enhancing password security based on the recommendations provided by the toadally_safe web application. By following these steps, you will learn practical strategies to strengthen your passwords and improve overall security.

4.1 Analyzing Feedback

Let's start by analyzing the feedback provided by toadally_safe, this well help us to identify weaknesses and vulnerabilities in your passwords.

4.1.1 Input a current password you would like to test and take a screenshot of the output.

This could be a password you currently use for an online account or a password you previously tested in the lab. By testing your own password, you can evaluate its strength and receive feedback on its potential weaknesses.

Review the feedback and suggestions provided by toadally_safe for your password. Take note of any specific weaknesses or vulnerabilities that are identified. These weaknesses may include factors such as password length, lack of character diversity, predictable patterns, or the usage of common words or phrases.

Pay close attention to the weaknesses highlighted by toadally_safe, as they indicate areas where your password's security can be enhanced. Understanding these weaknesses will enable you to make informed decisions on how to strengthen your password effectively.

Identify patterns or common mistakes in your passwords that can be addressed to enhance their security. Look for recurring themes or characteristics in your passwords that may make them vulnerable to guessing or brute-force attacks. This could include the use of personal information, easily guessable sequences, or common substitutions.

By analyzing the feedback provided by toadally_safe and identifying patterns or common mistakes in your passwords, you can take targeted actions to enhance their security. The insights

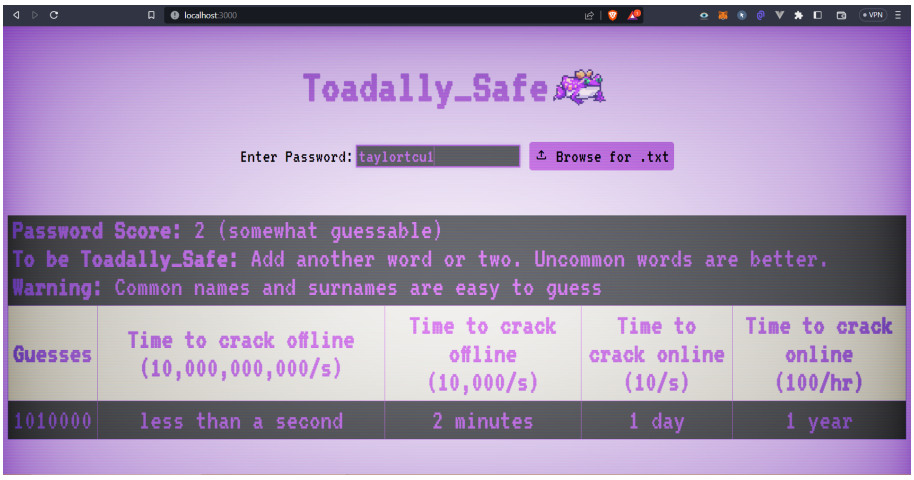


Figure 2: a

gained from this analysis will inform the strategies you employ to strengthen your passwords and mitigate potential vulnerabilities.

4.2 Increasing Password Length

When evaluating the strength and security of a password, password length is very important. Longer passwords are generally harder for attackers to guess or crack.

To crack passwords, attackers generally employ two methods: **brute-force attacks** and **dictionary attacks**. Brute-force attacks require *systematically testing all possible character combinations until the proper one is discovered.* Dictionary attacks, on the other hand, involve *guessing the password using pre-existing lists of often used passwords and common terms.*

When you increase the password size, the number of combinations increases exponentially, making it significantly harder for an attacker to brute-force. The longer the password, the more time and computational resources are required in order to successfully crack the password.

For example, consider a brute-force attack, a password of only 4 characters would take 26^4 (456,976) guesses, however, if you increase your password to 14 characters, it would take 26^{14} (6.45×10^{19}) guesses.

4.2.1 Make your password 12-14 characters or longer, review the feedback. Did this increase your password strength score? What changes did you notice?

It is typically advised to use passwords with a length of anything from 12 to 14 characters and beyond due to the growing computational power accessible to cybercriminals and improvements in password cracking tools. It's important to realize that longer passwords by themselves do not

provide complete security. To maintain a high degree of security, it's also critical to combine length with other best practices, like utilizing uncommon and random character combinations, avoiding typical patterns, and regularly upgrading passwords.

4.3 Utilizing Character Diversity

In order to create a secure password, you should keep in mind your character diversity. By including a wide range of characters, you increase the complexity of the password, making your password less easy to guess or crack using common techniques.

A secure password should include a variety of lowercase, uppercase, numbers, and symbols. By expanding the different characters categories you use, you expand the list of possible combinations, making the attacks significantly harder to crack.

Consider our earlier example of a password with only 4 characters, which takes 26^4 guesses. Implementing lowercase, uppercase, and numbers alone ends up being:
26 (uppercase letters A-Z) + 26 (lowercase letters a-z) + 10 (digits 0-9) = 62^4 guesses, which ends up being 14,319,360 more guesses, and we haven't even factored in using special characters yet.

In a typical U.S QWERTY Keyboard, there are 32 special characters, that being:
(~ ‘ ! @ # $ % ^ & * ( ) _ - + = | { } [ ] : ; ” ’ < > , . ? /)

4.3.1 Using your modified password from 4.2.1, include or replace characters so that you have a combination of lowercase, uppercase, numbers, and special characters. Review the feedback. Did this increase your password strength score? What changes did you notice?

By utilizing these special characters, along with lowercase, uppercase, and numbers in our example password, this now increases the amount of guesses to 94^4, which is 63,298,560 more guesses than without using them.

4.4 Avoiding Common Words or Phrases

The last aspect of creating a secure password is to avoid using common words or phrases. Since attackers often use dictionary attacks, they systematically use common passwords or words in attempt to guess your password, this means by using common sequences like 'abc', '123', or 'password', for example, you're putting your security at major risk.

The reason why you should avoid these common words is because they significantly reduce the strength of your password. Using common words makes your password more susceptible to guessing, even if the other features we talked about, length and character diversity, are implemented. For

example, "Password123!" is still not much of an improvement even if it does contain appropriate length and character diversity, because it contains common phrases.

Ideally, to create a strong and secure password, you should choose a combination of characters that don't form recognizable words. You can accomplish this by choosing random letters, numbers, and special characters that don't have a discernible meaning. The more unique and random the combination, the more resilient it becomes to attacks.

4.4.1 Using your modified password from 4.3.1, see if toadally_safe picks up any common phrases, can you yourself identify any common phrases? If your password contains any recognizable phrases, replace them with various characters. Did your password strength score increase? Take a screenshot of the output.

In conclusion, a strong password is one that cannot be easily guessed, even when confronted with advanced attacks. As a result, it is critical to avoid common terms or phrases, expand your password length to 12 characters or more, and incorporate character diversity.

4.4.2 Compare the screenshots from the beginning to the one you just took. Were you able to increase the strength of your password by following these steps? Have you already implemented a few of these techniques in your passwords?

5 Sources and Additional Reading

(1) zxcvbn github
https://github.com/dropbox/zxcvbn

(2) toadally_safe github
https://github.com/imtaylorgriffin/toadally_safe

(3) Password security: Complexity vs. length [updated 2021] - Infosec Resources.
https://resources.infosecinstitute.com/topic/password-security-complexity-vs-length/

(4) Password security + 10 password safety tips | Norton.
https://us.norton.com/blog/privacy/password-security

(5) How long should my password be? | Bitwarden Blog.
https://bitwarden.com/blog/how-long-should-my-password-be/

(6) How long should your password be? The data behind a safe ... - LMG Security.
https://www.lmgsecurity.com/how-long-should-your-password-be-a-technical-guide-to-a-safe-password-

length-policy/

(7) Use passwords with a length of anything from 12 to 14 characters
https://www.mass.gov/guides/password-best-practices-and-recommendations

6 Troubleshooting

Common issues and ways to fix them

6.0.1 Need to reset the container?

Make sure you are in the toadally_safe folder:

```
docker-compose up --build --remove-orphans --force-recreate
# Forcefully recreates everything
```